

Programmering i Python - Plotting og matematiske funksjonar

Funksjonsdefinisjonen

Lat oss sjå på tredjegradsfunksjonen

$$f(x) = 3x^3 + x^2 - 3.$$

Me kan definera ein python-funksjon som evaluerer funksjonen $f(x)$, som fylgjer.

In [1]:

```
1 def f(x): return 3*x**3 + x**2 - 3
```

Legg merke til bruken av aritmetiske operatorar, og `**` for potens.

Me kan testa denne funksjonen i python:

In [2]:

```
1 f(0.5)
```

Out[2]:

-2.375

Merk at me ikkje utan vidare kan få fleire verdier frå same blokk.

In [3]:

```
1 f(-1)
2 f(+1)
```

Out[3]:

1

Jupyter viser berre den siste verdien i blokken. Dersom me skal testa fleire verdier og skriva ut meir informasjon, kan me bruka `print`-funksjonen:

In [4]:

```
1 print ("f(-1) =", f(-1))
2 print ("f( 0) =", f(0))
3 print ("f(+1) =", f(1))
```

f(-1) = -5

f(0) = -3

f(+1) = 1

Me har sett tre ulike typar kodeliner over:

1. Definisjon `def f(x)` : funksjonen $f(x)$ vert definert, men ingenting vert skrive ut.
2. Evaluering `f(0.5)` på eiga line: funksjonen vert evaluert og me ser resultatet.
3. Kommando `print` : python får ein kommando, som i dette tilfellet skriv ut argumenta. Merk at dersom kodeblokken inneheld fleire evalueringar, vert berre den siste vist:

In [5]:

```
1 f(1)
2 f(0)
```

Out[5]:

-3

Me bruker `print` får å få kontroll over visinga.

Ein kommando og ei evaluering er eigentleg det same i python. Skilnaden er at `f(x)` gjev ein returverdi som python kan bruka vidare, men `print(x)` skriv på skjermen utan å gje ein returverdi. Det siste er ein sokalla bieffekt (*side effect*). Det er mogleg for ein funksjon å gje både returverdi og bieffekt, men mange reknar det som dårleg praksis.

Variablar og tilordning

Ein av dei mest fundamentale setningane i programmering er tilordning av variablar:

In [6]:

```
1 y = f(-2)
```

OK. Noko er kanskje skjedd her, men me veit ikkje kva. Lat oss sjå.

In [7]:

```
1 y
```

Out[7]:

-23

Det er viktig å merka seg at notasjonen `x = y` i python ikkje er eit utsagn som kan vera sant eller usant (samanlikning). Det er ein imperativ som seier at noko skal skje (tilordning). Etter tilordninga har `x` (variabelen på venstre side) fått ein ny verdi.

I Jupyter skal ein òg leggja merkje til at ein kan køyra blokkane i feil rekkjefylgje. Variablar vert tilordna når blokken vert køyrd. Difor er det ikkje sikkert at variablane har den verdien det ser ut som om dei har når ein les dokumentet som det er skrive. Lat oss sjå på eit døme.

Lat oss fyrst tilordna:

In [8]:

```
1 a = 1
```

So kan me bruka variabelen, t.d.

In [9]:

```
1 f(a)
```

Out[9]:

1

Her burde $f(a) = 1$. Lat oss no endra verdien på a .

In [10]:

```
1 a = 2
2 f(a)
```

Out[10]:

25

No bør $f(a) = 25$ men legg merke til at den forrige utrekninga ikkje er endra no. Me kan derimot gå tilbake og rekna den forrige blokken på nytt. Dersom me gjer det (utan å køyra tilordninga på nytt), får me $f(a)$ ogso der.

For å vera sikker på at alt er rekna ut i riktig rekkjefylgje kan ein køyra heile dokumentet i rekkjefylgje. *Spol fram*-knappen (dobbel pilhovud til høgre) i Jupyter gjer dette.

Tips: Det er ein god vane, når ein lurar litt på korleis ein programkonstruksjon verkar, å prøva seg fram. Skriv nokre enkle døme og prøv å køyra dei.

Ein lærer meir av døme enn av reglar.

Merknad Tilordninga `a=1` er òg ein kommando. Me kommanderer python til å endra verdien på `a`. Som me ser over gjev ikkje tilordninga nokon returverdi.

Plotting

I utgangspunktet er der berre nokre få kjernefunksjonar definert i python. Ei lang rekkje funksjonar for spesielle bruksområde finst i bibliotek som kan importerast etter behov. Der finst mange slike bibliotek, ofte fleire konkurrerande bibliotek for same bruksområde.

Til plotting skal me bruka matplotlib:

In [11]:

```
1 from matplotlib import pyplot as plt
```

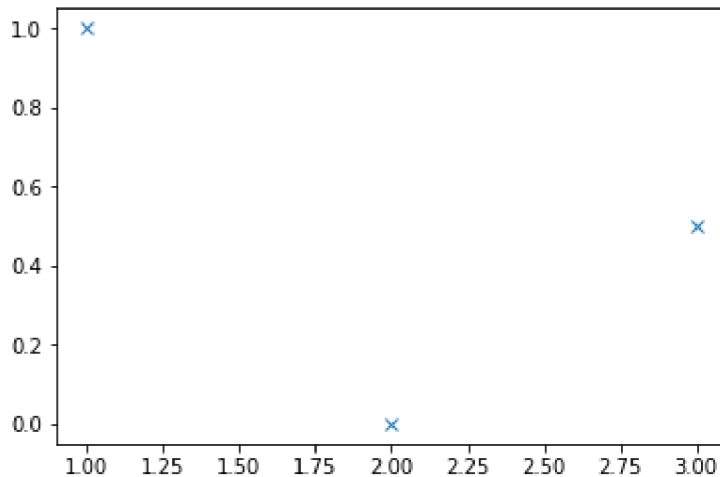
Her importerer me underbiblioteket `pyplot` og gjev det kortnamnet `plt`. I `pyplot` finn me funksjonar for plotting, t.d.

In [12]:

```
1 x = [ 1, 2, 3 ]
2 y = [ 1, 0, 0.5 ]
3 plt.plot(x,y, 'x')
```

Out[12]:

[<matplotlib.lines.Line2D at 0x7f41078b8f28>]



Dei tre argumenta til plot (over), er vektorar med x- og y-verdiar og ein streng som seier korleis punkta skal sjå ut.

Oppgåve

Når du gjer oppgåvene under kan du velja om du vil gå opp og endra koden i dømet over, eller kopiera og laga nye python-blokkar med nye døme.

1. Prøv å byta ut strengen 'x' med andre punktspesifikasjonar:

- '.'
- 'o-'
- 'r+:' Kva skjer?

2. Legg til punktet (1,5; 2) i plottet.

Listekomprehensjon

In [13]:

```
1 x = range(0,5)
2 print(x)
```

range(0, 5)

Funksjonen `range` gjev ein iterator. Me kan tenkja på det som ei liste med tal, frå 0 til 4 i dette tilfellet, men python reknar ikkje ut tala før det trengst, og `print` viser difor ikkje kva tal me får, berre korleis fylgja er definert.

Me kan laga ei liste i staden, vha. listekomprehensjon.

In [14]:

```
1 x = [ i for i in range(0,5) ]
2 print(x)
```

[0, 1, 2, 3, 4]

Listekomprehensjon er eit kraftig verktøy og enkel notasjon for å rekna ut talfølgjer, ta t.d.

In [15]:

```
1 y = [ 2*i+1 for i in x ]
2 print(y)
```

[1, 3, 5, 7, 9]

Legg merke til parallellen mellom listekomprehensjon her, og mengdekomprehensjon i matematikken. Matematisk ville eg skriva definisjonen over som

$$y = [2i + 1 | i \in x]$$

Dette er litt uvanleg notasjon, sidan me sjelden arbeider med ordna lister i matematikken. Hadde x og y vore uordna mengder ville me skriva

$$y = \{2i + 1 | i \in x\}$$

Funksjonsplott

Lat oss sjå korleis me kan plotta funksjons $f(x)$ med plot-funksjonen som me har prøvd. Fyrst må me definera x - og y -verdiar til plottet.

- som x -verdiar kan me bruka (heil)tala i intervallet $[-10, 10)$
- y -verdian må me rekna ut for kvar x -verdi.

In [16]:

```
1 x = range(-10,10)
2 y = [ f(i) for i in x ]
3 print( "x:", x )
4 print ( "x:", [i for i in x ] )
5 print( "y:", y )
```

x: range(-10, 10)

x: [-10, -9, -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

y: [-2903, -2109, -1475, -983, -615, -353, -179, -75, -23, -5, -3, 1, 25, 87, 205, 397, 681, 1075, 1597, 2265]

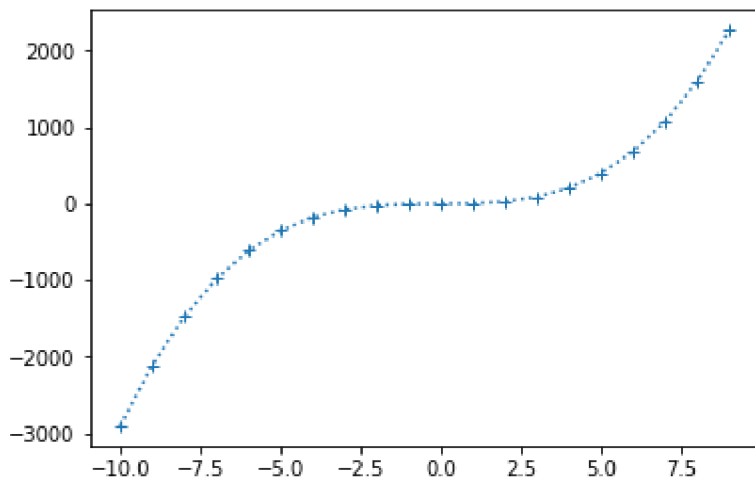
Då er me klare til å plotta.

In [17]:

```
1 plt.plot(x,y,'+:')
```

Out[17]:

```
[<matplotlib.lines.Line2D at 0x7f4107832f28>]
```



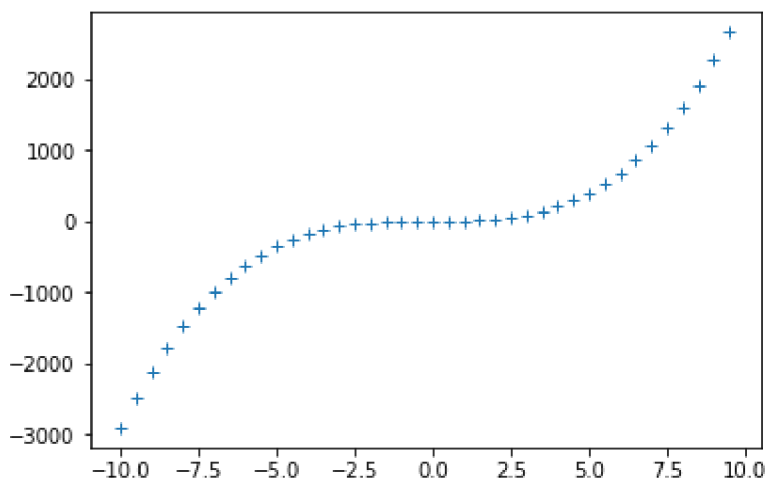
Det er sjølvsagt nyttig å kunna plotta med tettare x -verdiar enn berre heiltalsverdiar. Korleis kan me gjera det enkelt? Me kan i alle fall kombinera `range()` og *list comprehension*.

In [18]:

```
1 x = [ i*0.5 for i in range(-20,20)]  
2 y = [f(i) for i in x]  
3 plt.plot(x,y,'+')
```

Out[18]:

```
[<matplotlib.lines.Line2D at 0x7f41077f8eb8>]
```

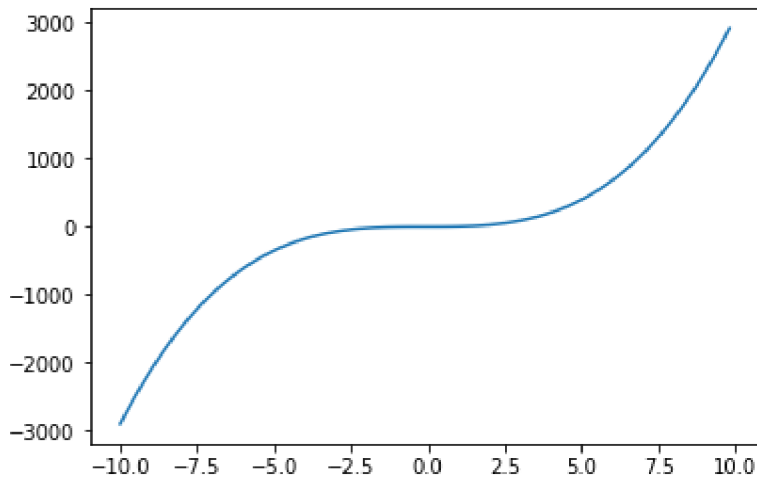


In [21]:

```
1 x = [ i*0.2 for i in range(-50,50)]  
2 y = [f(i) for i in x]  
3 plt.plot(x,y, '-')  
4 plt.axes()
```

Out[21]:

<matplotlib.axes._subplots.AxesSubplot at 0x7f4107753978>



Oppg ve

Tilpass x -verdiane og lag eit plot som tydleg viser tredjegradsformen med tre nullpunkt, topp- og botnpunkt.

Visuell tilpassing

Aksefors

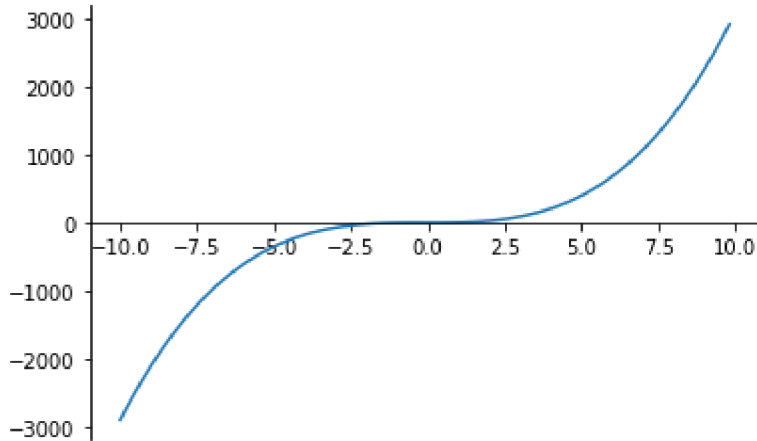
Mange er vande med   sj  plottet i forhold til linene $x = 0$ og $y = 0$, heller enn innramma i ein boks som pyplot gjer i utgangspunktet. Her er pyplot sv ert fleksibelt. Alt er mogleg, til gjengjeld krev det nokre liner   seia akkurat kva me ynskjer. Lat oss fyrst flytta x -aksen.

In [29]:

```

1 plt.plot(x,y)
2 ax = plt.gca()
3 ax.spines['top'].set_color('none')
4 ax.xaxis.set_ticks_position('bottom')
5 ax.spines['bottom'].set_position(('data',0))
6

```



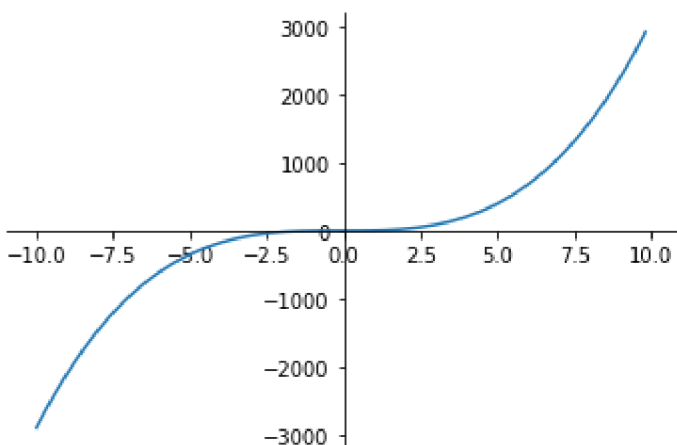
Me kan gjera det same med y-aksen. Merk at me må definera heile plottet i ein boks i jupyter. Me kan ikkje modifisera eit eksisterande plott.

In [30]:

```

1 plt.plot(x,y)
2 ax = plt.gca()
3 ax.spines['top'].set_color('none')
4 ax.xaxis.set_ticks_position('bottom')
5 ax.spines['bottom'].set_position(('data',0))
6 ax.spines['right'].set_color('none')
7 ax.yaxis.set_ticks_position('left')
8 ax.spines['left'].set_position(('data',0))
9
10

```



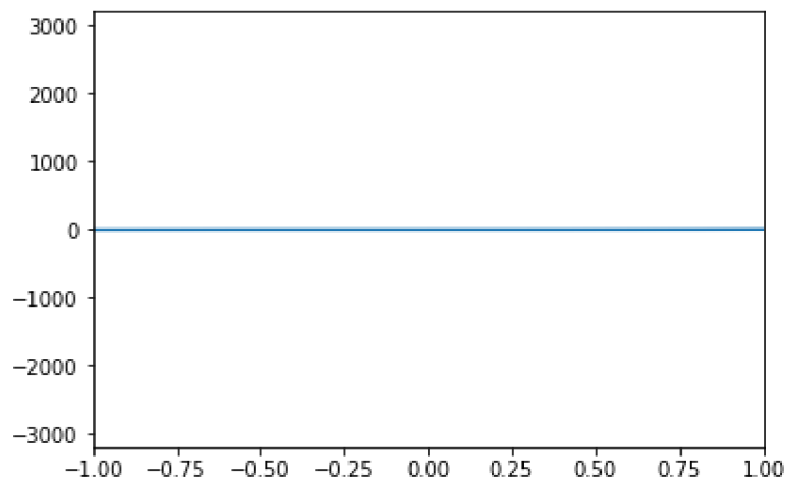
Definisjonsområde

In [33]:

```
1 plt.plot(x,y)
2 plt.xlim(-1,1)
```

Out[33]:

(-1, 1)

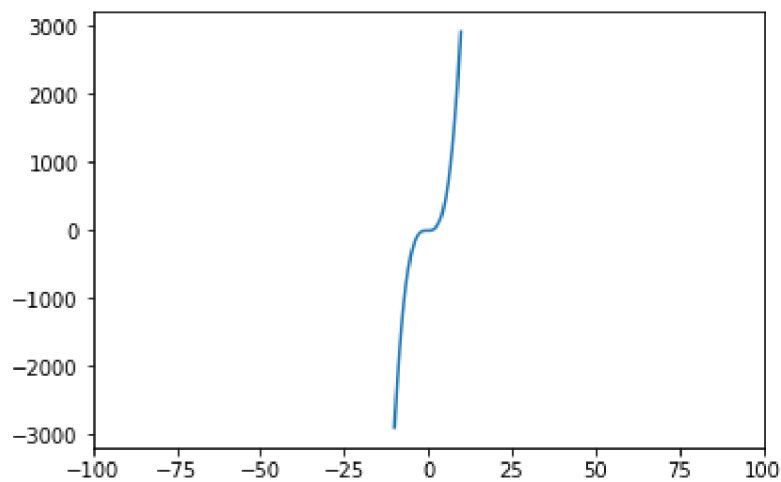


In [32]:

```
1 plt.plot(x,y)
2 plt.xlim(-100,100),
```

Out[32]:

(-100, 100)

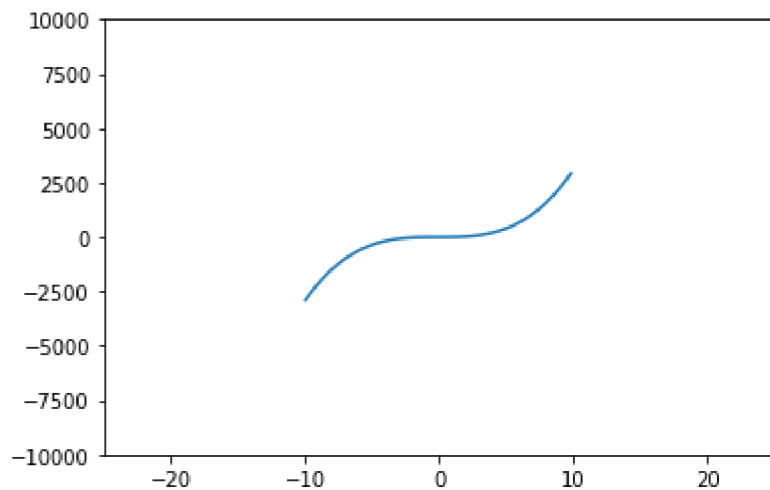


In [35]:

```
1 plt.plot(x,y)
2 plt.xlim(-25,25)
3 plt.ylim(-10000,10000)
```

Out[35]:

(-10000, 10000)



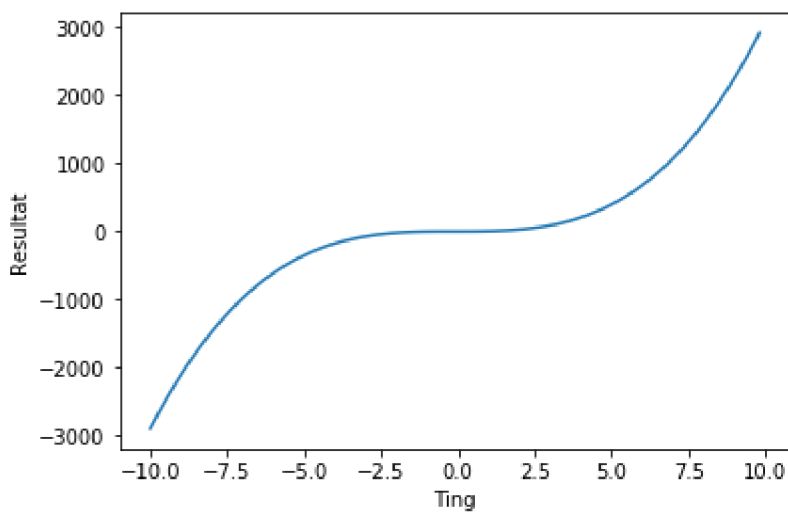
Tekst og forklaring

In [36]:

```
1 plt.plot(x,y)
2 plt.xlabel("Ting")
3 plt.ylabel("Resultat")
```

Out[36]:

Text(0, 0.5, 'Resultat')



Oppsummering

I denne økta har me lært:

- Generelle teknikkar med python-syntaks
 1. Funksjonsdefinisjon
 2. Tilordning ($x =$)
 3. Kommando: `print()`
 4. *list comprehension*
- Numeriske funksjonar i Python
 1. plot-funksjonen i matplotlib

Dokumentasjon

Der er mange kjelder om python. Dei varierer både i kvalitet og i formål, og det er difor ein god idé og søkja på litt på måfå, og vurderer kvaliteten sjølve. Rådet mitt er likevel å prøva seg fram *fyrst* og slå opp når du prøver på noko nytt.

- [StackOverflow](https://stackoverflow.com/) (<https://stackoverflow.com/>), har svært mange gode døme på løysingar på konkrete problem. Ein finn gjerne oppslag via [google](http://www.google.no) (<http://www.google.no>), men det er greitt å vera merksam på treff frå StackOverflow.
- [Scipy-Lectures](http://scipy-lectures.org/intro/matplotlib/matplotlib.html) (<http://scipy-lectures.org/intro/matplotlib/matplotlib.html>), gjev ein god gjennomgang av korleis ein kan tilpassa visualiseringa av plott
- Dei offisielle sidene er interessante og autoritative, men er ikkje den enklaste plassen å byrja når ein kan lite frå før. Dei inkluderer
 - [Matplotlib](https://matplotlib.org/) (<https://matplotlib.org/>),
 - [Numpy](https://www.numpy.org/) (<https://www.numpy.org/>),
 - [Scipy](https://www.scipy.org/) (<https://www.scipy.org/>),
 - [Python](https://www.python.org/) (<https://www.python.org/>).